

Normalização de Dados — CriptoInvest

Documento de análise das formas normais (**1FN**, **2FN** e **3FN**) aplicadas à modelagem relacional atual do CriptoInvest, conforme o schema definido em [criptoinvest_ddl.sql](#).

A análise é feita **tabela a tabela** sobre as 10 tabelas do modelo, sem nenhuma alteração no schema — o objetivo é demonstrar que o modelo já está integralmente normalizado até a Terceira Forma Normal (3FN).

1. Conceitos das Formas Normais

Forma Normal	Regra	Pergunta que precisa ser respondida "sim"
1FN — Primeira Forma Normal	Todos os atributos são atômicos (indivisíveis) e não existem grupos repetitivos nem atributos multivalorados. Cada célula contém um único valor e cada tabela tem chave primária.	Todos os campos guardam um único valor escalar?
2FN — Segunda Forma Normal	Está em 1FN e não há dependências parciais : todo atributo não-chave depende da chave primária inteira , e não apenas de parte dela. (Só é um risco real quando a PK é composta.)	Nenhum atributo depende de apenas parte da chave?
3FN — Terceira Forma Normal	Está em 2FN e não há dependências transitivas : nenhum atributo não-chave depende de outro atributo não-chave. Todo atributo não-chave depende direta e exclusivamente da chave primária.	Nenhum atributo não-chave determina outro atributo não-chave?

Observação de projeto: todas as PKs do modelo são **chaves substitutas (surrogate keys)** numéricas de coluna única (`id_*`, alimentadas por sequences). Esse padrão elimina por construção o risco de dependência parcial (2FN), pois uma PK de coluna única não tem "partes". As tabelas associativas (`posicao`, `alerta`) também usam PK substituta e protegem a regra de negócio N:N por meio de **chave única composta (UK)**, não da PK.

2. Análise Tabela a Tabela

2.1. `carteira` (pai da herança joined)

Item	Valor
PK	<code>id_carteira</code>
FKs	— (é a tabela raiz da herança)
Atributos	<code>descricao</code> , <code>saldo_reais</code> , <code>tipo</code>

- **1FN:** todos os atributos são atômicos — `descricao` (texto único), `saldo_reais` (numérico), `tipo` (discriminador `CHAR(2)` restrito a `'PF'` / `'PJ'`). Não há listas, conjuntos nem grupos repetitivos.
- **2FN:** PK de coluna única (`id_carteira`); não existe chave composta, logo não há possibilidade de dependência parcial. Todos os atributos dependem da carteira.
- **3FN:** `descricao` , `saldo_reais` e `tipo` descrevem diretamente a carteira e são independentes entre si — nenhum deriva de outro atributo não-chave. **Em 3FN.**

2.2. `carteira_pf` (filha — pessoa física)

Item	Valor
PK	<code>id_carteira_pf</code>
FK	<code>id_carteira_pf</code> → <code>carteira(id_carteira)</code> (PK = FK)
Atributos	<code>limite_diario_saque</code>

- **1FN:** `limite_diario_saque` é um único valor numérico — atômico.
- **2FN:** PK de coluna única; o único atributo próprio depende integralmente da PK.
- **3FN:** `limite_diario_saque` é um dado **específico de PF** que depende apenas da identidade da carteira PF, sem dependência transitiva. A decomposição da herança (ver §3) garante que ele **só existe quando faz sentido**, evitando colunas nulas na tabela pai. **Em 3FN.**

2.3. `carteira_pj` (filha — pessoa jurídica)

Item	Valor
PK	<code>id_carteira_pj</code>
FK	<code>id_carteira_pj</code> → <code>carteira(id_carteira)</code> (PK = FK)
Atributos	<code>regime_tributario</code>

- **1FN:** `regime_tributario` é atômico (`VARCHAR2(30)` restrito por `CHECK` a um conjunto fechado de valores).
- **2FN:** PK de coluna única; o atributo próprio depende da PK inteira.
- **3FN:** `regime_tributario` é específico de PJ e depende apenas da carteira PJ, sem dependência transitiva. **Em 3FN.**

2.4. `usuario`

Item	Valor
PK	<code>id_usuario</code>
FK	<code>id_carteira_pf</code> → <code>carteira_pf(id_carteira_pf)</code> (1:1, UK)
Atributos	<code>nome</code> , <code>email</code> , <code>senha</code> , <code>cpf</code> , <code>autenticacao_2fa</code>

- **1FN:** todos os atributos são escalares; `cpf` e `email` armazenam um único valor cada (não há "vários e-mails" numa coluna). Sem grupos repetitivos.
- **2FN:** PK de coluna única — sem dependência parcial.
- **3FN:** `nome`, `senha`, `cpf` e `autenticacao_2fa` descrevem diretamente o usuário. `email` e `cpf` possuem UK (são candidatas a chave), mas isso **não cria dependência transitiva**: nenhum atributo não-chave é determinado por outro atributo não-chave. A carteira PF do usuário é referenciada por FK (1:1), não copiada para dentro de `usuario`. **Em 3FN.**

2.5. empresa

Item	Valor
PK	<code>id_empresa</code>
FKs	<code>id_usuario</code> → <code>usuario(id_usuario)</code> (dono, 1:N); <code>id_carteira_pj</code> → <code>carteira_pj(id_carteira_pj)</code> (1:1, UK)
Atributos	<code>nome</code> , <code>cnpj</code>

- **1FN:** `nome` e `cnpj` são atômicos; `cnpj` guarda um único documento.
- **2FN:** PK de coluna única — sem dependência parcial.
- **3FN:** `nome` e `cnpj` descrevem diretamente a empresa. O dono é guardado como FK (`id_usuario`), e não pela cópia de dados do usuário (nome, cpf etc.), evitando dependência transitiva. A carteira PJ é igualmente referenciada por FK. **Em 3FN.**

2.6. criptoativo

Item	Valor
PK	<code>id_cripto</code>
FKs	—
Atributos	<code>nome</code> , <code>sigla</code> , <code>preco_atual</code> , <code>variacao_24h</code> , <code>categoria</code>

- **1FN:** todos os atributos são escalares; `sigla` e `categoria` são valores únicos.
- **2FN:** PK de coluna única — sem dependência parcial.
- **3FN:** `nome`, `sigla`, `preco_atual`, `variacao_24h` e `categoria` dependem diretamente do criptoativo identificado por `id_cripto`. `sigla` tem UK (chave candidata), mas nenhum atributo não-chave é funcionalmente determinado por outro atributo não-chave. **Em 3FN.**

2.7. transacao

Item	Valor
PK	<code>id_transacao</code>
FKs	<code>id_carteira</code> → <code>carteira(id_carteira)</code> (polimórfica PF/PJ); <code>id_cripto</code> → <code>criptoativo(id_cripto)</code>
Atributos	<code>tipo</code> , <code>quantidade</code> , <code>preco_unitario</code> , <code>taxa</code> , <code>data_operacao</code>

- **1FN:** cada transação é um evento único com atributos atômicos; não há lista de itens dentro de uma linha.
- **2FN:** PK de coluna única (`id_transacao`) — embora a transação relacione carteira e cripto, a **identidade** é a chave substituta, então não há dependência parcial de uma chave composta.
- **3FN:** `tipo`, `quantidade`, `preco_unitario` e `data_operacao` descrevem o evento. A `taxa` é **derivada** (`valor_bruto * 0,1%`) e calculada na aplicação, sendo persistida como o valor histórico efetivamente cobrado no momento da operação — é um fato da transação, dependente apenas dela. Dados do criptoativo (nome, sigla) **não** são copiados para cá; são alcançados via FK `id_cripto`, evitando dependência transitiva. **Em 3FN.**

2.8. relatorio

Item	Valor
PK	<code>id_relatorio</code>
FK	<code>id_carteira</code> → <code>carteira(id_carteira)</code>
Atributos	<code>data_geracao</code> , <code>valor_total_carteira</code> , <code>total_investido</code> , <code>total_vendido</code> , <code>total_taxas</code> , <code>lucro_total</code> , <code>rentabilidade_percentual</code>

- **1FN:** todos os valores são numéricos/escalares atômicos.
- **2FN:** PK de coluna única — sem dependência parcial.
- **3FN:** o relatório é um **snapshot histórico** — uma fotografia consolidada da carteira numa `data_geracao`. Seus números são agregados calculados e **congelados** naquele instante; dependem do par (relatório, momento de geração) representado pela PK, e **não** de recálculo a partir de outras tabelas. Por isso a coexistência de `lucro_total` e `rentabilidade_percentual` no mesmo registro é intencional (histórico imutável) e não fere a 3FN, pois ambos são propriedades registradas do próprio snapshot, não derivações de outro atributo não-chave vivo. **Em 3FN.**

2.9. `posicao` — associativa N:N (`carteira` × `criptoativo`)

Item	Valor
PK	<code>id_posicao</code> (substituta)
UK composta	<code>(id_carteira, id_cripto)</code> — <code>uk_posicao_carteira_cripto</code>
FKs	<code>id_carteira</code> → <code>carteira(id_carteira)</code> ; <code>id_cripto</code> → <code>criptoativo(id_cripto)</code>
Atributos	<code>quantidade_atual</code> , <code>preco_medio_compra</code> , <code>data_primeira_aquisicao</code> , <code>data_ultima_atualizacao</code>

- **1FN:** resolve o relacionamento N:N entre carteira e criptoativo de forma **atômica**: cada linha representa **uma** posição (um par carteira-cripto), com saldo agregado em colunas escalares. Sem a tabela associativa, seria necessário um campo multivalorado ("vários criptoativos numa carteira"), o que **violaria a 1FN** — a decomposição em `posicao` é justamente o que garante a atomicidade.
- **2FN:** a PK é a chave substituta de coluna única `id_posicao` ; portanto não há dependência parcial. A unicidade do par é garantida pela **UK composta** `(id_carteira, id_cripto)` , que protege a regra de negócio (uma única posição por par) sem transformar essas colunas na PK. Os atributos `quantidade_atual` , `preco_medio_compra` e as datas dependem da posição como um todo.
- **3FN:** todos os atributos próprios são fatos agregados da posição (saldo consolidado e datas) e não derivam uns dos outros. Os dados da carteira e do criptoativo permanecem nas suas tabelas, acessados por FK. **Em 3FN.**

2.10. `alerta` — associativa N:N (`usuario` × `criptoativo`)

Item	Valor
PK	<code>id_alerta</code> (substituta)
FKs	<code>id_usuario</code> → <code>usuario(id_usuario)</code> ; <code>id_cripto</code> → <code>criptoativo(id_cripto)</code>
Atributos	<code>limite_variacao</code> , <code>ativado</code> , <code>data_configuracao</code>

- **1FN:** resolve o N:N entre usuário e criptoativo de forma atômica: cada linha é **um** alerta de um usuário sobre um criptoativo, com atributos escalares. Evita o grupo repetitivo "vários alertas dentro de usuário", preservando a 1FN.
- **2FN:** PK de coluna única `id_alerta` — sem dependência parcial. (Diferente de `posicao` , o modelo permite intencionalmente mais de um alerta por par usuário-cripto, por isso **não** há UK composta aqui.)
- **3FN:** `limite_variacao` , `ativado` e `data_configuracao` são propriedades diretas do alerta e independentes entre si; dados de usuário e criptoativo são referenciados por FK, sem cópia. **Em 3FN.**

3. Herança Joined e sua Contribuição para a Normalização

A herança de `carteira` foi implementada na estratégia **joined** (uma tabela por classe): `carteira` (pai, atributos comuns + discriminador `tipo`), `carteira_pf` (atributo exclusivo de PF) e `carteira_pj` (atributo exclusivo de PJ), ligadas por **PK = FK**.

Essa decomposição **reforça a 3FN** porque:

- **Evita atributos opcionais/nulos por natureza:** `limite_diario_saque` só existe para PF e `regime_tributario` só existe para PJ. Numa tabela única, uma das duas colunas estaria sempre nula conforme o tipo, e cada atributo dependeria do valor do discriminador `tipo` (um atributo não-chave) — exatamente uma **dependência transitiva**. Separando em `carteira_pf` / `carteira_pj`, cada atributo passa a depender apenas da PK da respectiva tabela.
- **Mantém a integridade polimórfica:** `transacao`, `posicao` e `relatorio` referenciam a tabela pai `carteira`, então a FK funciona igual para PF e PJ sem duplicar estrutura.

4. Conclusão

Todas as 10 tabelas do modelo CriptoInvest possuem:

1. atributos **atômicos** e sem grupos repetitivos → **1FN**;
2. chaves primárias **substitutas de coluna única** (e UK composta apenas onde a regra de negócio exige), eliminando **dependências parciais** → **2FN**;
3. atributos não-chave que dependem **direta e exclusivamente** da chave primária, com relacionamentos resolvidos por **FK** e **tabelas associativas** (`posicao`, `alerta`) e com a herança decomposta em **joined**, eliminando **dependências transitivas** → **3FN**.

Portanto, o modelo relacional do CriptoInvest encontra-se na Terceira Forma Normal (3FN).